Quantization · Pruning Day 1

양자화의 수학 — 숫자를 INT8 에 담기

HYUNDAI AI Insight Campus · 온디바이스 AI · Quantization·Pruning 1/6

2026. 11. 16 (월) 09:00–18:00 · 8교시 · Colab 또는 Kaggle (하드웨어 없음)

FP32 → INT8 | scale · zp | 대칭 · 비대칭 | per-channel | clipping | INT32 누적

강사 : 김정인 · 김구수

오늘의 학습 목표

수업이 끝나면 여러분은 다음을 할 수 있습니다

1 왜 경량화인가

모델 크기 = 파라미터 × byte. FP32 → INT8 이면 1/4 (SimpleConvNet 1.74 → 0.43 MB)

2 숫자 표현

FP16 최대 65,504 · INT8 −128~127, round → clip → astype 순서를 지킨다

3 scale · zero-point

calc_scale_zp · quantize · dequantize 를 만들고 오차 ⩽ scale/2 를 확인한다

4 대칭 · per-channel

ReLU 출력은 비대칭 , weight 는 대칭 · per-channel 이 유리함을 실측한다

5 clipping

범위를 좁히는 trade-off 를 MSE 곡선으로 보고 , outlier 에 대응한다

6 정수 연산 · INT8 CNN

INT8 × INT8 → INT32 로 fc2 를 계산하고 , 손으로 INT8 SimpleConvNet 을 만든다

시간표

8 교시 × 50 분 · 점심 12:00–13:00 · 매 교시 보기 → 따라하기 → 바꿔보기

교시	시간	주제	형태	애니메이션
1	09:00–09:50	왜 경량화인가 — 크기 · 메모리 · 속도 · 전력	강의 + 실습	D1-1 모델 다이어트
2	10:00–10:50	숫자 표현 — FP32 · FP16 · INT8 · UINT8	강의 + 실습	D1-2 비트로 숫자 담기
3	11:00–11:50	Affine 양자화 — scale · zero-point	강의 + 실습	D1-3 scale · zp 조절기
4	13:00–13:50	대칭 vs 비대칭 — ReLU 출력과 weight	강의 + 실습	D1-4 대칭 · 비대칭 비교
5	14:00–14:50	Granularity — per-tensor vs per-channel	강의 + 실습	D1-5 per-tensor vs channel
6	15:00–15:50	Clipping 과 Outlier — 범위 정하기	강의 + 실습	D1-6 clip 범위 슬라이더
7	16:00–16:50	정수 연산 추론 — INT8 × INT8 → INT32	강의 + 실습	D1-7 정수 행렬곱
8	17:00–17:50	미니 프로젝트 — INT8 SimpleConvNet · 채점	개인 실습	D1-8 INT8 CNN 조립

오늘의 자료 · 온라인 링크

예측 (애니메이션) → 직접 구현 (노트북) → 점검 (복습 퀴즈) — 모두 브라우저에서 바로 열립니다

기본 주소 (Day 1)

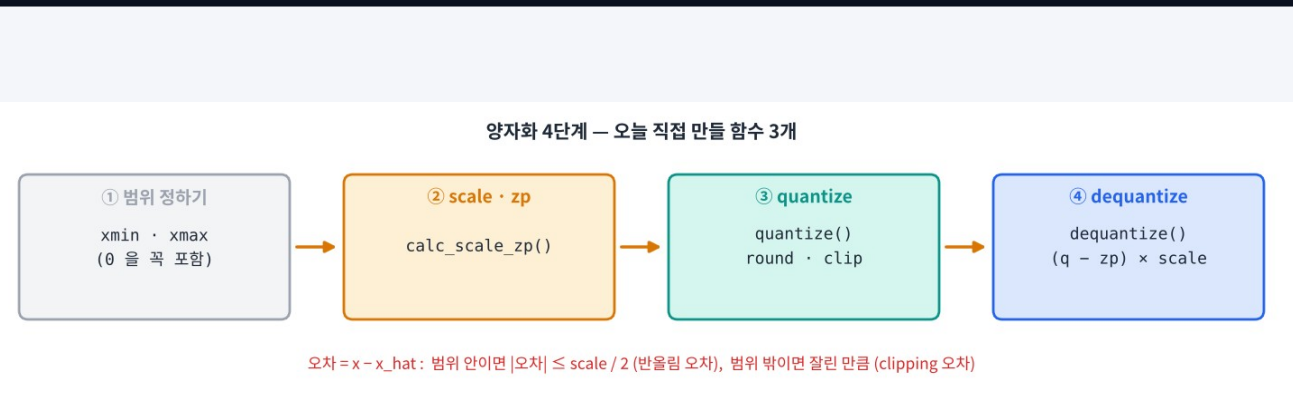
https://imguru-mooc.github.io/AI_CAMPUS/QP-day1/

animation.html	애니메이션 8 개 (D1-1 ~ D1-8) — #d1-1 처럼 바로가기
review_quiz.html	Day 1 복습 퀴즈 — 수업이 끝난 뒤 24 문항 (9 가지 유형) · 채점 후 제출
QP_Day1_lecture.ipynb	강의 노트북 — 힌트를 보고 직접 구현 29 개 + 미니 프로젝트 (Colab · Kaggle)
QP_Day1.pdf	오늘 슬라이드 PDF
data/mnist_cnn.pt · mnist_sample.npz	학습된 SimpleConvNet 가중치 · MNIST 100 장 (노트북이 자동으로 내려받음)
my_quant.py (오늘 만드는 파일)	8 교시 과제 1 — calc_scale_zp · quantize · dequantize · sym_scale → Day 2 에서 import

오늘의 큰 그림 — 양자화 4 단계를 진짜 가중치로

범위 → scale · zp → quantize → dequantize · 오차를 재고 , 방식을 바꿔 비교한다





오늘 쓰는 데이터 — 모두 SimpleConvNet 실측

파라미터 — 1 교시

433,890 · FP32 1.74 MB

conv weight — 4·5 교시

(30, 1, 5, 5) · −0.482 ~ 0.387

conv 활성값 A — 4·6 교시

432,000 개 · 0 ~ 2.137 · 0 이 52%

fc2 + 은닉 H — 7 교시

(10, 100) · 100 장 × 100

weight 3 개 — 8 교시

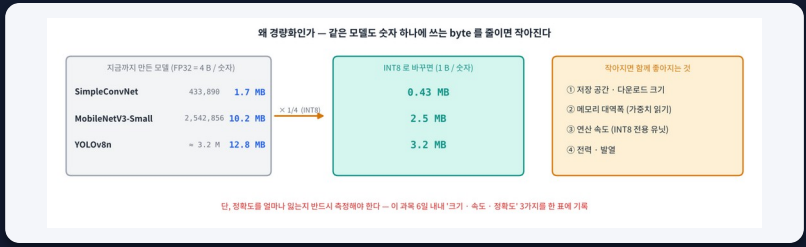
conv · fc1 · fc2 fake quant

1 왜 경량화인가 — 크기 · 메모리 · 속도

09:00–09:50

학습 목표

- 크기 = 파라미터 수 × 숫자 하나의 byte
- FP32 4 · FP16 2 · INT8 1 · INT4 0.5
- 작아지면 저장 · 대역폭 · 연산 · 전력 이득
- 크기 1/4 ≠ 속도 4 배 — 항상 측정



✍️ 직접 구현 (힌트 → 구현 → ✅ 확인)

1. 구현 1-1 파라미터 433,890 · 1.736 MB
2. 구현 1-2 세 모델 × 3 자료형 표
3. 구현 1-3 INT8 압축률 4.0
4. 🔁 BITS 16 · 8 · 4 · 2

▶ 애니메이션 D1-1 모델 다이어트

핵심 개념

크기

```
n_params = sum(p.numel() for p in cnn.parameters())
fp32_mb = n_params * 4 / 1e6   # 1.736
```

세 모델

```
SimpleConvNet 1.74 → 0.43 MB
MobileNetV3-S 10.17 → 2.54 · YOLOv8n 12.78 → 3.19
```

4 가지



체크포인트

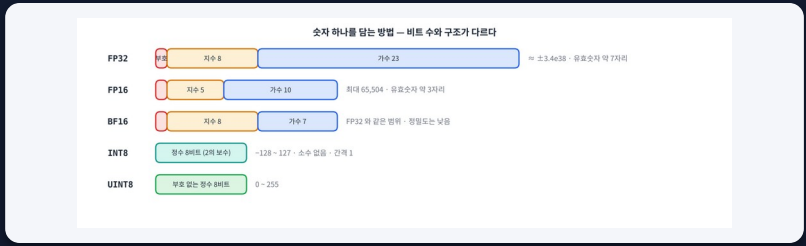
경량화 결과는 항상 크기 · 속도 · 정확도 세 가지를 한 표에 적는다 . 이 과목의 qp_baseline.csv 가 그 표다 .

2 숫자 표현 — FP32 · FP16 · INT8 · UINT8

10:00–10:50

학습 목표

- 부동소수점 = 부호 + 지수 (범위) + 가수 (정밀도)
- FP16 최대 65,504 · 유효숫자 약 3 자리
- INT8 −128~127 · UINT8 0~255 = 256 칸
- 정수 변환은 round → clip → astype



✍️ 직접 구현 (힌트 → 구현 → ✅ 확인)

1. 구현 2-1 np.iinfo 범위
2. 구현 2-2 FP16 반올림 · inf
3. 구현 2-3 round → clip → int8
4. 🔁 INT8 × INT8 = 16 ?

▶ 애니메이션 D1-2 비트로 숫자 담기

핵심 개념

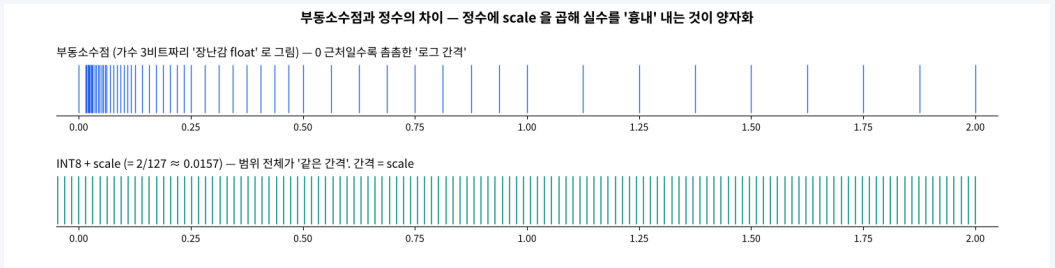
FP16

```
x.astype(np.float16)
# 0.1 → 0.09998 · 1000.1 → 1000 · 70000 → inf
```

INT8

```
[0.3, 1.7, 200].astype(np.int8)  # [0, 1, -56] ✗
np.clip(np.round(v), -128, 127).astype(np.int8)  # ✓
```

간격



체크포인트

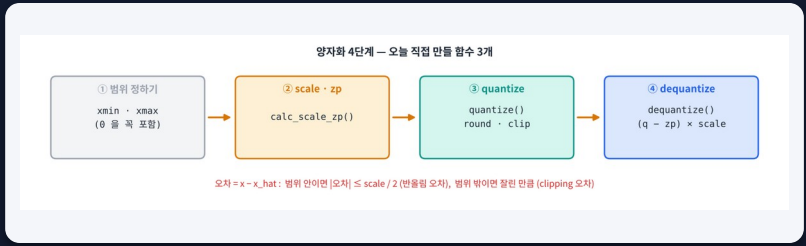
astype(np.int8) 은 소수를 버리고 범위를 넘으면 wrap-around 한다 — 오류 없이 틀린다 .

3 Affine 양자화 — scale · zero-point

11:00–11:50

학습 목표

- scale = (xmax − xmin) / (qmax − qmin)
- zp = qmin − round(xmin / scale) · 정수
- q = clip(round(x/s) + zp) · x̂ = (q − zp)·s
- 범위에 0 포함 → 0 은 정확히 0



✍️ 직접 구현 (힌트 → 구현 → ✅ 확인)

1. 구현 3-1 calc_scale_zp → 4/255, 64
2. 구현 3-2 quantize → [0,64,96,143,255]
3. 구현 3-3 dequantize · 오차 ≤ s/2
4. 구현 3-4 범위 밖 = clipping 오차

▶ 애니메이션 D1-3 scale · zp 조절기

핵심 개념

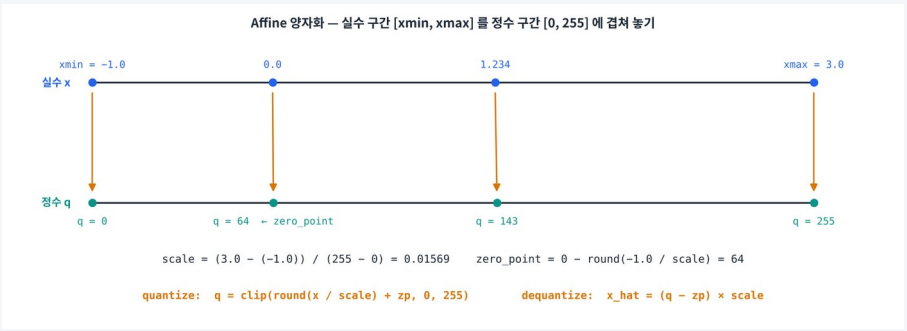
손계산

```
[−1, 3] → UINT8: scale 0.015686 · zp 64
1.234 → q 143 → x̂ 1.2392 ( 오차 0.0052)
```

두 오차

```
범위 안 : |오차| ≤ scale/2 = 0.0078
범위 밖 : 4.0 → 2.996 ( 오차 1.004)
```

매핑



체크포인트

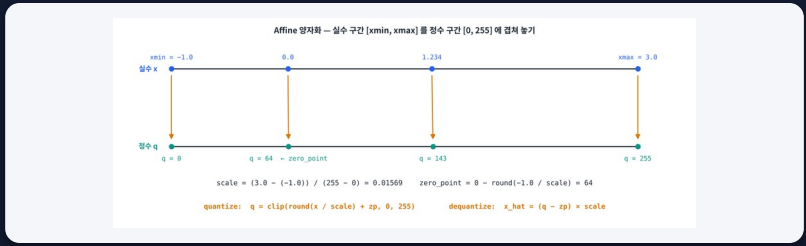
양자화에 필요한 숫자는 scale 과 zero_point 두 개뿐 . 계산은 INT32 그릇에서 , 저장만 8 비트로 .

4 대칭 vs 비대칭 — ReLU 출력과 weight

13:00–13:50

학습 목표

- 대칭 : scale = max|x| / 127 · zp = 0
- −127 ~ 127 만 사용 (−128 버림)
- ReLU 출력 → 비대칭 · weight → 대칭
- ORT 활성값 U8 · TensorRT 는 대칭



✍️ 직접 구현 (힌트 → 구현 → ✅ 확인)

1. 구현 4-1 sym_scale · sym_quantize
2. 구현 4-2 활성값 : 대칭 MSE 5.1 배
3. 구현 4-3 weight: 1.20 배
4. 🔁 SHIFT 0.3 → 2.55 배

▶ 애니메이션 D1-4 대칭 · 비대칭 비교

핵심 개념

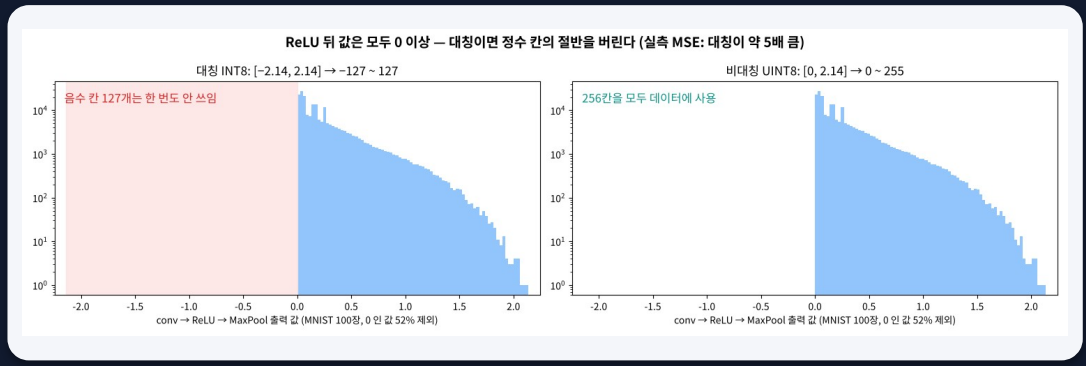
ReLU 출력

```
비대칭 scale 0.00838 · MSE 2.49e-06
대칭   scale 0.01683 · MSE 1.26e-05 (5.1 배)
```

weight

```
conv −0.482 ~ 0.387 → 대칭 손해 1.20 배
zp = 0 → 정수 행렬곱이 단순 (7 교시)
```

실측



체크포인트

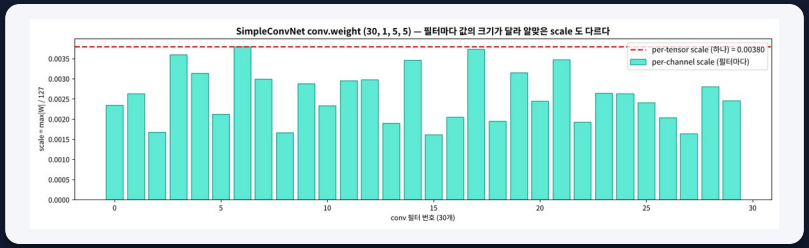
데이터가 한쪽으로 치우칠수록 비대칭이 유리 . weight 는 작은 손해 대신 zp = 0 의 단순함을 택한다 .

5 Granularity — per-tensor vs per-channel

14:00–14:50

학습 목표

- per-tensor: scale 1 개 — 큰 채널에 맞춰짐
- per-channel: 출력 채널 (0 번 축) 마다 scale
- scale 을 (C, 1, 1, 1) 로 broadcast
- 활성값은 보통 per-tensor



✍️ 직접 구현 (힌트 → 구현 → ✅ 확인)

1. 구현 5-1 per-tensor scale 0.003798
2. 구현 5-2 per-channel (30,)
3. 구현 5-3 MSE 1.91 배 정확
4. 구현 5-4 fc1 에도 → 2.63 배

▶ 애니메이션 D1-5 per-tensor vs channel

핵심 개념

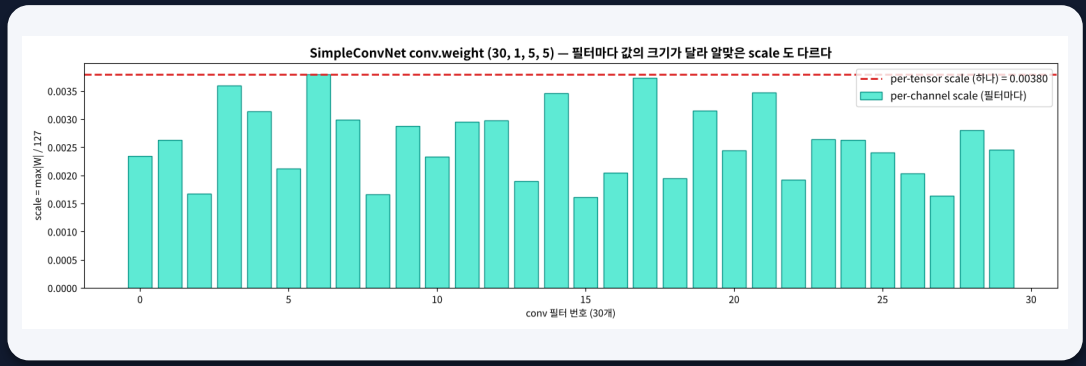
채널 scale

```
sc = np.abs(W).reshape(30, -1).max(axis=1) / 127
sc4 = sc[:, None, None, None]   # (30, 1, 1, 1)
```

튀는 필터

```
필터 0 ×10 → per-tensor MSE 4.43e-05
per-channel 2.47e-06 → 18 배 차이
```

필터 30 개



체크포인트

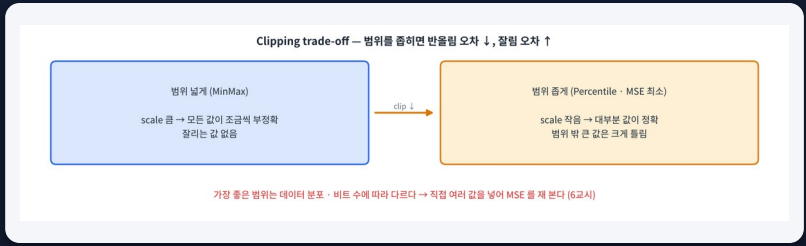
weight 의 표준은 per-channel. 비트가 적을수록 (4 · 2 bit) 그 이득이 더 커진다 — 8 교시에서 확인 .

6 Clipping 과 Outlier — 범위 정하기

15:00–15:50

학습 목표

- 오차 = 반올림 (범위 안) + 잘림 (범위 밖)
- 범위 ↓ → 반올림 ↓ · 잘림 ↑
- 최적 clip 은 분포 · 비트 수에 따라 다름
- MinMax · Percentile · Entropy · MSE



✍️ 직접 구현 (힌트 → 구현 → ✅ 확인)

1. 구현 6-1 99.9% = 1.607 (0.1% 넘음)
2. 구현 6-2 최적 clip 8b 0.97 · 4b 0.71
3. 구현 6-3 outlier: 지표가 결론을 바꿈
4. 🔁 백분위 99 · 99.9 · 99.99

▶ 애니메이션 D1-6 clip 범위 슬라이더

핵심 개념

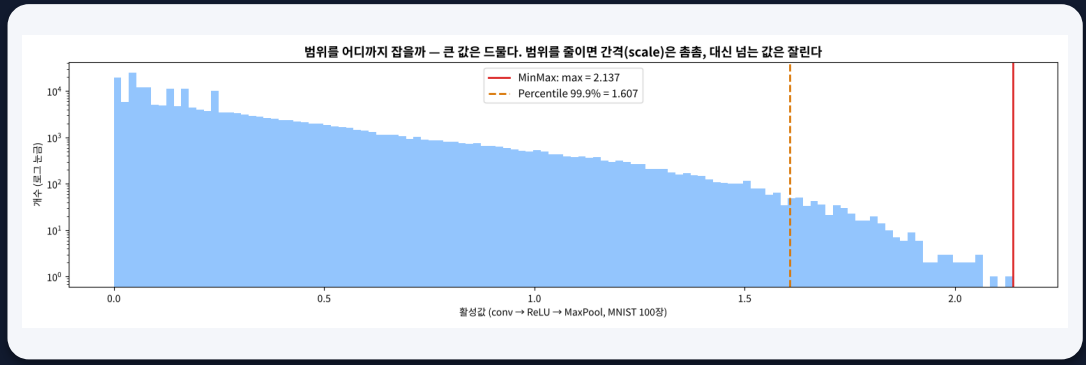
sweep

```
clips = np.linspace(0.05, 1, 96) * A.max()
best = clips[np.argmin([mse_clip(A, c, bits) ...])]
```

outlier

```
max × 20 하나 : 나머지 값은 Percentile 42 배 정확
전체 MSE 는 MinMax 가 낮음 — 지표를 먼저 정한다
```

분포



체크포인트

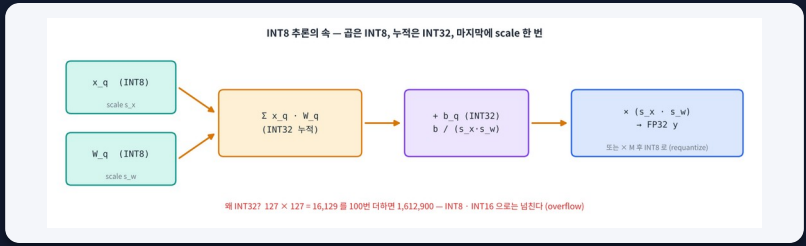
Percentile 이 항상 낫지는 않다 . 실제 calibration 은 후보 범위를 모델 정확도로 최종 확인한다 (Day 2).

7 정수 연산 추론 — INT8 × INT8 → INT32

16:00–16:50

학습 목표

- y ≈ s_w s_x (W_q x_q + b_q)
  - 곱 INT8 · 누적 INT32 · bias INT32
  - 실수 곱셈은 마지막 한 번
- requantize: M = s_x s_w / s_y



✍️ 직접 구현 (힌트 → 구현 → ✅ 확인)

1. 구현 7-1 INT32 누적 · ±65,000
2. 구현 7-2 b_q = round(b / (s_x s_w))
3. 구현 7-3 최대 차이 0.10 · 일치 1.00
4. 구현 7-4 M = 0.001924

▶ 애니메이션 D1-7 정수 행렬곱

핵심 개념

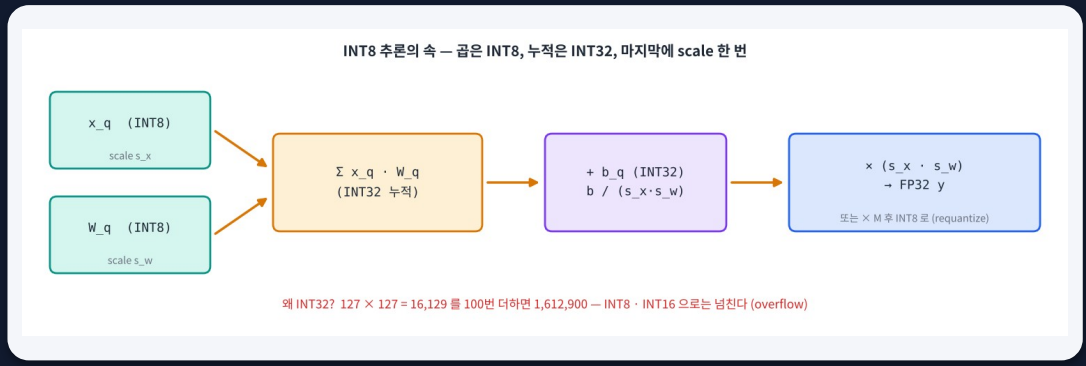
fc2

```
acc = xq.astype(np.int32) @ Wq.astype(np.int32).T
y_int = (acc + bq) * (sx * sw)
```

비대칭이면

```
(x_q − zp) → − zp × (W 행의 합) 보정 항
→ weight 를 대칭으로 두는 이유
```

흐름



체크포인트

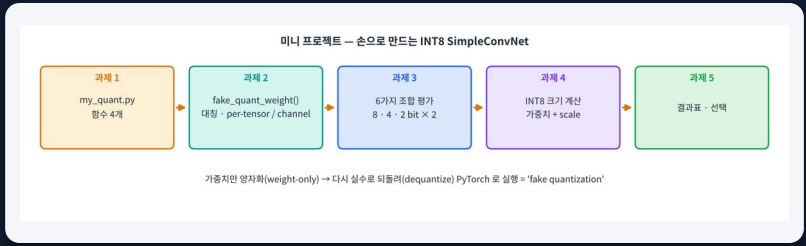
int8 @ int8 은 조용히 넘친다 . 반드시 INT32 로 넓혀 곱하고 , bias 는 INT32 로 둔다 .

8 미니 프로젝트 — 손으로 만드는 INT8 CNN

17:00–17:50

학습 목표

- fake quant = 양자화 → 역양자화 weight
- 8 · 4 · 2 bit × per-tensor / channel
  - 정확도 · 일치율 · max_diff · 크기
  - 선택 규칙 : 손실 ⩽ 1%p → 비트 ↓



✍️ 직접 구현 (힌트 → 구현 → ✅ 확인)

1. 과제 1 my_quant.py (20)
2. 과제 2 fake_quant_weight (20)
3. 과제 3 6 가지 조합 평가 (20)
4. 과제 4 · 5 · 6 크기 · 선택 · 해석

▶ 애니메이션 D1-8 INT8 CNN 조립

핵심 개념

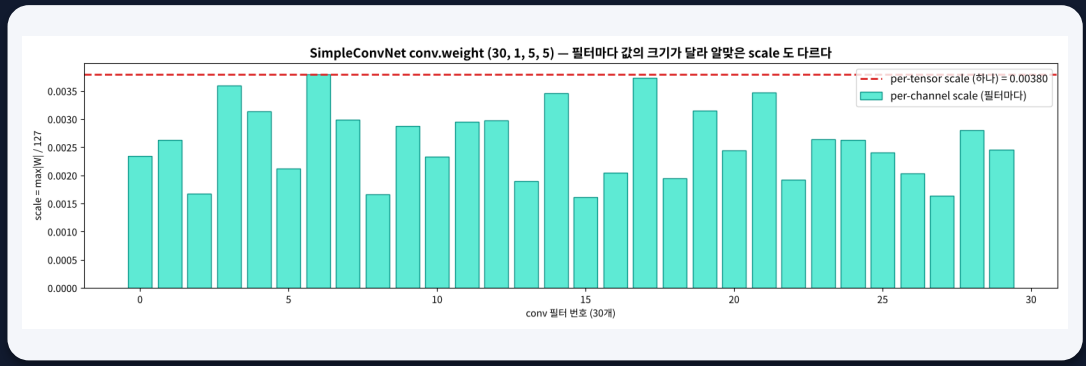
결과

```
8b: 일치 1.00 · max_diff 0.067
4b: 1.59 / 1.32 · 2b: 정확도 0.25 → 0.52 (per-channel)
```

크기

```
433,750 + scale 140×4 + bias 140×4 = 434,870 B
FP32 1,735,560 B 의 1/3.99
```

필터 scale



체크포인트

자동 채점 90 점 + 과제 6 해석 10 점 . 채점 셀 결과가 보이는 상태로 노트북을 공유해 제출한다 .

오늘의 핵심 정리

각 항목 옆 번호는 다시 볼 애니메이션 · 정리가 끝나면 review_quiz.html 로 Day 1 복습 퀴즈를 풀고 제출

경량화	크기 = 파라미터 × byte · FP32 → INT8 = 1/4 · 크기 · 속도 · 정확도를 함께 측정	D1-1
숫자 표현	FP16 최대 65,504 · INT8 256 칸 · round → clip → astype	D1-2
scale · zp	$q = \mathrm{clip}(\mathrm{round}(x/s) + zp)$ · $\hat{x} = (q - zp) \cdot s$ · 범위 안 오차 ≤ s/2 · 0 포함	D1-3
대칭 · 비대칭	ReLU 출력은 비대칭 (5.1 배) · weight 는 대칭 (1.2 배 , zp = 0)	D1-4
per-channel	출력 채널마다 scale · (C, 1, 1, 1) broadcast · conv 1.91 배 · fc1 2.63 배	D1-5
clipping	반올림 ↔ 잘림 trade-off · 최적 clip 은 비트 · 분포에 따라 · 지표를 먼저 정한다	D1-6
정수 연산	INT8 곱 → INT32 누적 · bias INT32 · × s_x s_w 한 번 · requantize M	D1-7
INT8 CNN	8b 거의 무손실 · 비트 ↓ 일수록 per-channel 중요 · 파일 = weight + scale + bias	D1-8

Day 2 로 가는 다리 — ONNX Runtime PTQ

오늘 손으로 한 일을 ONNX Runtime 이 한 줄로 — my_quant.py 로 검산한다

Day 1 (오늘)		Day 2 (11/17 화)
calc_scale_zp · quantize	→	ORT 가 만든 scale · zp 를 my_quant 로 검산
대칭 · 비대칭	→	QDQ vs QOperator · S8 / U8
per-channel	→	quantize_static(per_channel=True)
MinMax · Percentile clip	→	CalibrationMethod 비교 (+ Entropy)
INT32 누적 정수 연산	→	INT8 ConvInteger · QLinearConv
fake quant SimpleConvNet	→	mnv3s_int8.onnx · qp_baseline.csv

범위 → scale · zp → quantize → dequantize — 내일은 이 4 단계를 ONNX 그래프 안의 QuantizeLinear · DequantizeLinear 노드로 만난다